Project 2 ROS: UAV Behaviour, Control, and Kalman Filter Estimation

Arnav Salkade A0273649N
Patrick Joy Surbakti A0266288M
Harsh Ahuja A0312072W

April 7, 2026

1 Introduction

This report presents the design, implementation, and tuning of the UAV estimation and control stack developed for Project 2. The system consists of three main components: a behavior node for high-level mission-state transitions, a lookahead-based proportional controller for path following, and a Kalman filter for state estimation. Particular attention is given to z-axis estimation, since altitude errors were found to have a direct impact on tracking stability and landing performance. The report also examines how estimator tuning influenced controller behaviour, showing that reliable flight performance depended on both accurate sensor fusion and well-tuned control gains.

2 Behavior Node

The behavior node was implemented closely according to the project pseudocode and is responsible for managing the UAV mission states. Its role is to coordinate takeoff, tracking, return, and landing by updating the active waypoint and triggering state transitions based on waypoint completion and turtlebot motion. In addition, the node continuously requests a fresh plan from the UAV's current pose to the current target waypoint so that the path remains consistent with the latest mission state.

2.1 callbackTimer_()

At the start of the function, we decided to use the function `reachedWaypoint_()` to check whether the drone is sufficiently close to its current desired waypoint. If the waypoint has been reached, the function decides whether to remain in the current state or move the next state based on the state of the turtlebot. The transition logic can be listed as:

- In TAKEOFF state, the drone proceeds to INITIAL if the turtlebot has stopped, else it will proceeds to TURTLE_POSITION state. This will make sure to prepare the drone for landing only when the turtlebot has stopped
- In INITIAL state, the drone proceeds to land only if the turtlebot has stopped, else it will start tracking the turtlebot by entering TURTLE_POSITION state
- In TURTLE_POSITION state, the drone transitions to TURTLE_WAYPOINT

- In TURTLE_WAYPOINT state, the drone transitions to INITIAL
- In LANDING, the drone moves to END, and stop the mission.

After transition check, the function reads the current position of the drone from `odom_`. If the current state is `TURTLE_POSITION` and a valid plan exists for the turtlebot, the waypoint will be updated to follow the turtle's latest known position. This ensures the drone follows the live position of the turtlebot instead of a prerecorded position at the start of state transition. In the end, `requestPlan_()` is called to continuously request a trajectory from the drone's current pose to the active waypoint and update the drone's path.

2.2 transition_(int new_state)

Whenever the transition function _ is called, it updates the drone `state_` to `new_state`. Essentially, the transition only sets the corresponding goal for the drone during state transition according to the new states:

- TAKEOFF: The robot is enabled by publishing `true` to `pub_enable_`, and the waypoint is set to the takeoff position at cruising altitude. This causes the drone to rise from its initial ground position to its operating height.
- INITIAL: The waypoint is set to the initial reference position at cruising altitude.
- TURTLE_POSITION: The waypoint is set to the first pose in `turtle_plan_` at cruising altitude.
- TURTLE_WAYPOINT: The waypoint is set to the last pose in `turtle_plan_` at cruising altitude.
- LANDING: The waypoint is set to the initial ground position so that the drone descends and lands at its starting location.
- END: The robot is disabled by publishing `false`, the timer is cancelled, and the mission ends.

2.3 reachedWaypoint_()

This function checks whether the drone has arrived at its current waypoint by calculating the difference between the drone's current position and the target waypoint position in x, y, z directions.

3 Controller Node

A proportial (P) controller was implemented to follow the chosen lookahead point along the planned path. The position error between the drone and the lookahead point is first transformed into the drone body frame. The transformed errors are then multiplied by proportional gain coefficients to generate velocity commands in the horizontal and vertical directions. If the resulting horizontal velocity is outside the allowable range, the velocity in x and y will be scaled by $\frac{max\_xy\_vel\_}{xy\_speed}$. As the drone approaches the lookahead point, the position error decreases, and hence the commanded velocity also decreases. This prevents overshooting, especially when chasing after the turtlebot. The proportional gains determine how strongly the drone responds to the position error, the higher the gain, the faster the drone response to the change and it increase the chance of overshooting and oscillation. On the other hand, lower gain will make the drone motion smoother yet more sluggish,

and thus might not be able to catch up with the turtlebot. The proportional gain coefficient for horizontal velocity `kp_xy_` are tested between 0.5 to 1.0 and tuned to 0.7, while the z gain was set at 4 to reduce the chance of vertical overshooting or oscillations and maintain z estimation accuracy. Additionally, the required parameters such as the drone rotating 30 deg/s clockwise at all time has also been implemented. The final tuned controller gains and estimator covariance values used in the implemented system are summarised in Table 1.

Table 1: Final tuned controller and estimator parameters

Parameter	**Value**	**Purpose**
`kp_xy_`	0.7	Horizontal path-following gain
`kp_z_`	4.0	Vertical tracking gain
GPS variance in x	0.009873171	Horizontal state correction
GPS variance in y	0.012809	Horizontal state correction
GPS variance in z	0.005	Vertical correction
Sonar variance	0.00868	Vertical correction
Barometer variance	0.211	Vertical correction

4 Kalman Filter Estimation

The Kalman Filter is a robust estimation scheme to update the values of certain state variables based on the generation of a correction that creates consistency with tuned sensor measurements and maximises the probability of such a consistent correction given a known or assumed probability field surrounding any estimation made. The structure of the Kalman Filter used to update the current drone consists of a unified IMU-based prediction step for x,y,z and yaw values. The z-axis was estimated most accurately and precisely, and was corrected at a high frequency by both sonar readings and GPS readings. X and Y were updated only by noisy GPS measurements, and yaw was singularly updated by a magnetometer.

The standard estimator models the vertical motion using a two-state vector, but was updated with a bias term due to the observation of a small z-axis offset at stasis:

$$X_z = \begin{bmatrix} z \\ \dot{z} \\ b \end{bmatrix}.$$

During prediction, the state is propagated with a constant-acceleration model:

$$X_z^{k+1} = F_z X_z^k + W_z a_z,$$

where

$$F_z = \begin{bmatrix} 1 & \Delta t & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad W_z = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \\ 0 \end{bmatrix}.$$

The vertical acceleration is taken from the IMU after subtracting gravity. In the code, the covariance update for the prediction step is

$$P_z^{k+1} = F_z P_z^k F_z^T + \sigma_{\text{imu},z}^2 (W_z W_z^T),$$

where $\sigma_{\text{imu},z}^2$ is represented by `var_imu_z_`.

This update model is structurally replicated within the Magnetometer yaw update, GPS x,y,z updates and barometric z update. H above is formulated as such in order to append bias to the innovation scalar, for which reason its 3rd component is 1.

The following documents the x,y coupled prediction and measurement model for x and y coordinates.

$$F_{xy} = \begin{bmatrix} 1 & \Delta t & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad , W_x = \begin{bmatrix} \frac{1}{2}\Delta t^2 \cos(\theta) & -\frac{1}{2}\Delta t^2 \sin(\theta) \\ \Delta t \cos(\theta) & -\Delta t \sin(\theta) \\ 0 & 0 \end{bmatrix}, \qquad W_y = \begin{bmatrix} \frac{1}{2}\Delta t^2 \sin(\theta) & \frac{1}{2}\Delta t^2 \cos(\theta) \\ \Delta t \sin(\theta) & \Delta t \cos(\theta) \\ 0 & 0 \end{bmatrix}.$$

The horizontal state estimates are propagated using the IMU-derived input u_{xy}, while the associated covariance matrices are updated using the process noise covariance Q.

$$Q = \begin{bmatrix} \sigma^2_{\text{imu},x} & 0 \\ 0 & \sigma^2_{\text{imu},y} \end{bmatrix}$$

$$X_x^{k+1} = F_{xy} X_x^k + W_x u_{xy}, \qquad X_y^{k+1} = F_{xy} X_y^k + W_y u_{xy},$$

$$P_x^{k+1} = F_{xy} P_x^k F_{xy}^T + W_x Q W_x^T, \qquad P_y^{k+1} = F_{xy} P_y^k F_{xy}^T + W_y Q W_y^T.$$

for state vectors

$$X_x = \begin{bmatrix} x \\ \dot{x} \\ b_x \end{bmatrix}, \qquad X_y = \begin{bmatrix} y \\ \dot{y} \\ b_y \end{bmatrix}.$$

Yaw prediction was structured as follows, communicating that the angular velocity at every next timestep was predicted to be the angular velocity measurement at that time, and yaw angle was updated with the product of timestep size and measured angular velocity (approximately 30 degrees per second for this project, in which case angular acceleration is 0 and hence the following linear formulation holds):

$$F_{yaw} = \begin{bmatrix} 1 & \Delta t \\ 0 & 0 \end{bmatrix}, \qquad W_{yaw} = \begin{bmatrix} dt \\ 1 \end{bmatrix}$$

$$\psi^{k+1} = F_{yaw} \psi^k + W_{yaw} u_\psi$$

$$P_\psi^{k+1} = F_{yaw} P_\psi^k F_{yaw}^T + W_{yaw} \sigma^2_{imu,\psi} W_{yaw}^T$$

Finally, the covariance matrix must be made symmetrical manually after each update loop, as sensor noise can disturb this symmetry. Symmetry is an important attribute as it is within the covariance matrix's definition to be symmetrical. We wish to do so without destroying the information encoded within covariance matrix, and may do so by simply averaging each element with its counterpart on the other side of the 3x3 matrix's diagonal (which represents the same covariance information on account of the commutativity of covariance. This can be achieved by adding P to its transpose, and dividing by 2, effectively achieving this operation. We implement this step after every update is completed.

$$P \leftarrow \frac{1}{2}(P + P^T).$$

5 Estimation and Sensor Fusion

When expanded, the update equations from any sensor-based correction callback demonstrates an approximately inverse relationship between sensor variance R and size of update step, on account of:

$$Update = \frac{(PH^T)(Innovation)}{R + HPH^T}$$

The latter product in the denominator is a scalar for z as H is a 1x3 matrix and P is a 3x3 matrix for all coordinates in our stack. This relationship thereby permits variance to encode the degree of "trust" any one sensor is given when updating a state vector. While the ideal case would be for these variances to be tuned to exactitude based on input csv data, this was seen to produce offsets > 0.8m in x and y estimations in simulation. Therefore all variances were tuned to adjust for this till said offset was reduced significantly.

5.1 Z estimation

The z-axis results reveal a clear trade-off between estimator smoothness and robustness. When the sonar covariance was set too low, the filter over-trusted sonar measurements and became sensitive to obstacle-induced disturbances, producing sharp dips in the estimated altitude. Increasing the sonar covariance reduced these transient dips, but excessive retuning also reduced the short-term smoothness previously provided by the sonar updates. The final covariance values therefore represent a compromise in which spurious sonar disturbances are suppressed without allowing GPS noise to dominate the altitude estimate.

When a valid sonar measurement is available, it is treated as a direct correction to the vertical state through the following measurement update:

$$H = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

$$K_{\text{sonar}} = P_z H^T (H P_z H^T + R_{\text{sonar}})^{-1},$$

$$X_z \leftarrow X_z + K_{\text{sonar}} \left(z_{\text{sonar}} - H X_z \right),$$

$$P_z \leftarrow (I - K_{\text{sonar}} H) P_z.$$

That no significant offsets were seen from the UAV's true z indicates that GPS and sonar-based z updates were equally reliable. During a trial run, however, the following z-data was collected, for barometric variance of 0.211 and sonar and GPS variance of approximately 0.0005.

These dips were observed in z due to disruptions in sonar data collection due to obstacles within the simulation environment. These dips would trigger the controller to induce oscillatory state progression till altitude estimation resettled, and would cause the drone to drift more vertically in the process. These offsets were eliminated by a retuning of the sonar covariance to about 100 times its original value. This however, led to sizable inaccuracies in z estimation, accompanied with significant reduction of the dips observed. to eliminate this error, the value was re-tuned to approximately 0.00868, with GPS variance maintained at 0.005, resulting in the estimations in Figure 2.

The erstwhile observed deviations were practically erased. However, the cleaner trend offered by high-trust sonar updates was mildly compromised due to the same, potentially due to the more prominent fluctuations in GPS-based data given the new variances.

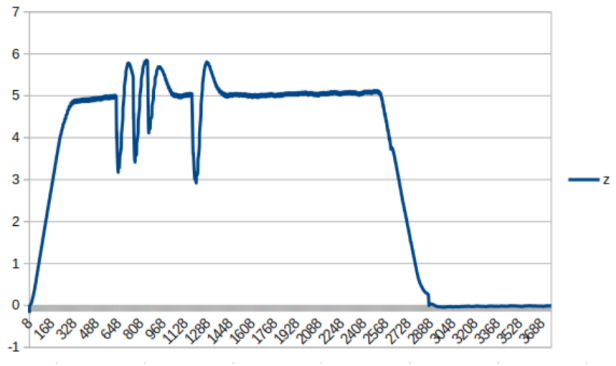

Figure 1: Z estimation data for a sonar+GPS dominant trial run

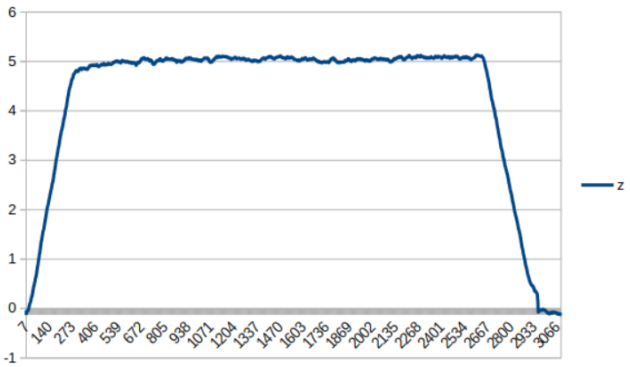

Figure 2: Z estimations with updated covariance values.

5.2 X and Y Estimations

In contrast to the z-axis, the x- and y-axis estimates were more difficult to stabilise because they relied primarily on GPS corrections, which were noisier and more susceptible to drift. As a result, the horizontal estimates exhibited larger fluctuations and weaker convergence than the vertical estimate.

The x and y states were corrected mainly through GPS measurements, with the corresponding Kalman gain and observation matrices following the same formulation used in the other estimation updates. These equations are therefore omitted here for brevity. In practice, however, the GPS-based x and y estimates were frequently observed to drift away from the UAV's true position and did not naturally return to the correct value, as shown in Figure 3.

To reduce this error, the GPS measurement variances for x and y were retuned iteratively while observing the resulting estimation offset. The initial variance values were estimated from a `.csv` file containing post-estimation coordinate data recorded while the UAV was at stasis, since stationary data was more consistent and easier to analyse than measurements collected during flight. Starting from an initial variance of approximately 0.0004, the x and y variances were tuned to 0.009873171 and 0.012809, respectively.

This retuning reduced the overall horizontal estimation error, and the inclusion of bias states improved the mean estimate further. However, noticeable fluctuations around the UAV's true position still remained, indicating that GPS-only correction was insufficient for highly stable horizontal estimation. This also explains why the final improvements in x and y were more limited than those achieved in the z-axis estimation.

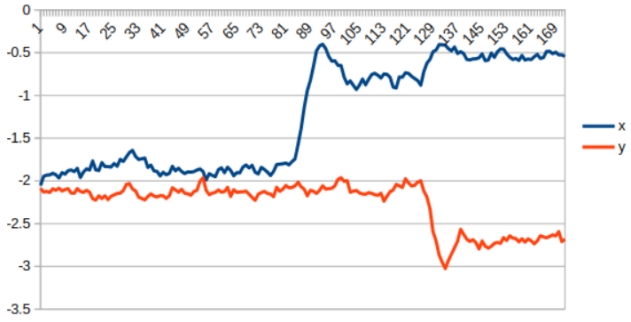

Figure 3: X and Y estimations without bias estimation

The effect of adding bias estimation to prediction and estimation, maintaining the same variance values, is as observed in Figure 4. While inaccuracy was reduced, the values still fluctuated quite significantly around the UAV's origin (-2,-2) at stasis.

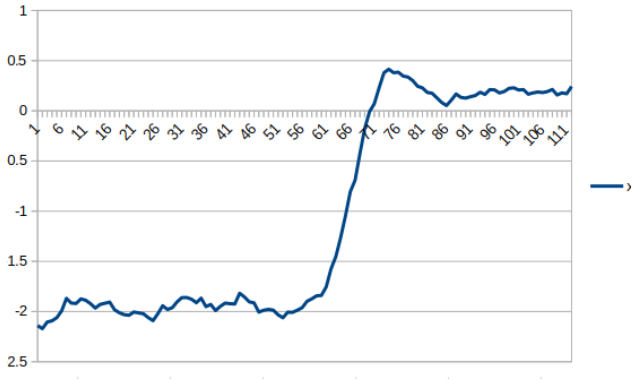

Figure 4: X estimations after bias involvement

Heuristic solutions to this problem were considered, wherein the primary challenge was directing the readjustment of X and Y values without knowledge of ground truth. One particular solution tested involved the tracking of a maximum and minimum coordinate value, allowing readjustment based on natural coordinate drift during motion, and offsetting these extrema backwards or forwards based on the sign of the product of matrix K and innovation as a potential indicator of the location of a more accurate estimate. However, since the updates scaled in proportion to the values of the extrema, this heuristic tended to generate very chaotically oscillating estimates, and it was finally excluded from the final stack.

5.3 Yaw Estimation

Yaw estimation was comparatively more stable than position estimation due to the consistent behaviour of the magnetometer measurements and is structured identical to other estimation processes. The yaw angle is derived as follows

$$\theta = -arctan(\frac{B_y}{B_x})$$

In order to ensure frame consistency, the resultant yaw angle is negated due to the z axis of the magnetometer pointing in the opposite direction to the world frame's z axis. Negligible to no yaw

deviations were observed at stasis, and so the measured magnetometer variance value was retained. Because the yaw estimate showed negligible steady-state error and no significant oscillatory behaviour during testing, additional retuning was not necessary. In contrast to the z-axis and horizontal position states, yaw estimation remained sufficiently stable under the measured magnetometer noise and therefore did not become a limiting factor in overall flight performance.

This also suggests that, unlike position estimation, the yaw estimation problem in this system was well-conditioned with respect to the available sensor data, and did not require further augmentation or bias correction.

5.4 Limitations

Despite the final tuning improvements, several limitations remained in the system. The sonar sensor was still susceptible to disturbance from obstacles in the simulation environment, which introduced transient errors in altitude estimation and occasionally triggered undesired controller responses.

In addition, the x- and y-axis estimates remained more difficult to stabilise because they relied primarily on GPS measurements, which were noisier and more prone to drift than the sensors used for z-axis estimation. Although variance tuning and bias estimation reduced the overall error, noticeable fluctuations around the UAV's true position persisted.

These observations indicate that the current estimation performance is limited by sensor quality and measurement reliability. Further improvements would likely require more robust outlier rejection mechanisms and additional sensing modalities to improve horizontal state estimation.

6 Conclusion

This project demonstrated the importance of designing the UAV behaviour logic, controller, and estimator as an integrated system rather than as separate modules. The behavior node successfully handled the mission sequence for takeoff, tracking, return, and landing, while the lookahead-based proportional controller provided a simple and effective method for path following.

Gain tuning revealed the expected trade-off between responsiveness and oscillation. Higher gains improved tracking speed but increased the risk of overshoot, while lower gains produced smoother but slower responses. The final controller values achieved stable tracking without excessive oscillation.

The most significant improvement came from Kalman filter tuning, particularly along the z-axis. By augmenting the vertical state with a bias term and retuning the sonar and GPS covariances, altitude estimation became more robust. This reduced oscillatory behaviour caused by unreliable sonar updates and improved overall flight stability.

In contrast, x- and y-axis estimation remained more challenging because these states depended primarily on noisier GPS measurements. Although bias estimation reduced the average error, noticeable horizontal fluctuations persisted, highlighting the limitations of GPS-only correction.

Overall, the final system achieved stable mission execution and demonstrated that reliable flight performance depends heavily on consistent sensor fusion. A key takeaway is that estimator tuning should be evaluated not only through estimation accuracy, but also through its downstream impact on controller behaviour.

Future improvements could include sonar outlier rejection, systematic covariance identification from logged data, and the integration of additional sensing modalities to improve horizontal state estimation.

Member Contribution

(Matric: A0273649N) contributed to the estimator, including Kalman filter implementation and variance tuning, and also supported the behavior and controller components. (Matric: A0312072W) contributed to further X and Y variance tuning and overall Project 2 implementation and system integration. (Matric: A0266288M) contributed to the behavior and controller implementation, as well as testing and validation of the required deliverables.